Privacy-Preserving Biometric Identification Using FHE: (with CKKS-Based Computations)

Raneem Ibraheem (ID: 212920896)
Aseel Nahas (ID: 212245096)

Supervised by: Prof. Adi Akavia, Fall 2024

January 26, 2025

Abstract

Biometric identification systems offer enhanced security through unique physiological traits like facial recognition, but their reliance on sensitive data raises critical privacy concerns. Fully Homomorphic Encryption (FHE) [4] addresses this by enabling computations on encrypted data, though challenges such as precision loss and computational overhead persist. This work implements an end-to-end privacy-preserving biometric identification system using the Cheon–Kim–Kim–Song (CKKS) [17] FHE scheme, evaluated on the *LFW-deepfunneled* dataset [27] with a transformer-based classifier and ArcFace embeddings [15].

The study first quantifies the impact of limited precision—simulating FHE-induced noise—on biometric identification accuracy. At **8 decimal precision**, the system achieves **85.52% accuracy** (where the accuracy is equals to the percentage of correct identifications) and **99.36% AUC** (where the AUC represents the ability to distinguish between genuine and impostor matches), degrading only to **84.59% accuracy** and **99.01% AUC** at **2 decimals**, validating robustness to noise. Precision truncation below 2 decimals causes sharper declines (e.g., **67.03% accuracy at 1 decimal**), highlighting the critical balance between privacy and utility.

Next, encrypted similarity computations using TenSEAL [19] achieve near-perfect alignment with cleartext results:

- **Euclidean Distance** [9]: Average error = 1.34×10^{-6} (max 1.61×10^{-6})
- **Cosine Similarity** [1]: Average error = 1.89×10^{-6} (max 2.19×10^{-6})

The end-to-end system processes **a progressing number of test samples (1, 5, 10, 20, 50) against all encrypted templates** with **100% Top-10 accuracy**, demonstrating practical viability. Homomorphic operations incur manageable overhead:

- **Encryption**: 4.8 ms/sample
- **Computation**: 67.20 s/sample
- **Ciphertext Size**: 0.32 MB/vector

These results confirm that CKKS-based FHE, combined with precision-aware training, enables secure biometric identification without compromising accuracy. The work bridges the gap between cryptographic privacy and machine learning efficacy, providing a blueprint for real-world deployment in authentication systems.

1 Introduction

Biometric identification systems are revolutionizing authentication by leveraging unique physiological traits such as facial features. However, their reliance on sensitive data raises critical privacy concerns, particularly when computations occur on untrusted platforms. While traditional encryption protects data at rest and in transit, it requires decryption for processing—a vulnerability that Fully Homomorphic Encryption (FHE) addresses by enabling computations directly on encrypted data. The Cheon–Kim–Kim–Song (CKKS) scheme, optimized for approximate arithmetic, offers a promising framework for privacy-preserving machine learning but introduces challenges such as precision loss and computational overhead. This work tackles these challenges through an end-to-end evaluation of a biometric identification system that integrates CKKS-based FHE with modern machine learning techniques.

1.1 Motivation and Challenges

The adoption of FHE in biometric systems hinges on two core requirements:

- **Accuracy Preservation:** Cryptographic operations like CKKS encryption introduce numerical noise, which can degrade the performance of biometric models reliant on high-precision computations (e.g., facial embeddings).
- **Practical Feasibility:** Homomorphic operations incur significant computational overhead, raising questions about scalability for real-world applications.

This study evaluates a facial recognition pipeline built on the *Labeled Faces in the Wild (LFW)* dataset, using ArcFace embeddings and a transformer-based classifier. We quantify the system's resilience to precision truncation—a proxy for FHE-induced noise—and implement encrypted similarity metrics to demonstrate privacy-preserving authentication.

1.2 Research Contributions

Our work makes the following contributions:

- **Robustness to Precision Loss:** We demonstrate that the biometric classifier retains 85% accuracy even when embeddings are truncated to 2 decimal places, a critical threshold for balancing privacy and utility in FHE settings.
- **Encrypted Similarity Metrics:** Using the TenSEAL library, we implement homomorphic Euclidean distance and cosine similarity computations with errors below 2×10^{-6}, achieving near-perfect alignment with cleartext results.

- **End-to-End System Evaluation:** The integrated system processes encrypted facial templates with 100% Top-10 accuracy, validating its practicality with a ciphertext size of 0.32 MB per template and computation times scalable to real-world deployments.

1.3 Key Findings

- The transformer-based classifier exhibited graceful degradation under precision truncation, maintaining 85.52% accuracy at 8 decimals and 84.59% at 2 decimal.
- Homomorphic similarity metrics achieved sub-micron errors (e.g., 1.34×10^{-6} for Euclidean distance), confirming CKKS's suitability for biometric comparisons.
- End-to-end encrypted identification required 70.21 s per sample, dominated by homomorphic dot product computations, with no loss in ranking consistency compared to cleartext.

1.4 Related Work

Prior studies have explored FHE for machine learning [16] and biometric identification [14], but gaps remain:

- **Precision-Accuracy Tradeoffs:** Existing work often assumes fixed precision levels; we systematically quantify how truncation impacts biometric accuracy.
- **System Integration:** While CKKS has been applied to neural networks [16], few works unify precision analysis, encrypted similarity metrics, and scalability testing in a single pipeline.

Our work bridges these gaps, demonstrating that CKKS-based FHE, when paired with precision-aware training, enables secure and accurate biometric identification—a critical step toward real-world adoption.

2 Technical Background

Biometric identification systems rely on extracting and comparing unique features from biological data, such as faces, fingerprints, or irises. This project focuses on face recognition using embeddings generated by deep learning models and evaluated using similarity metrics. Additionally, privacy-preserving computation is integrated using Fully Homomorphic Encryption (FHE) to enable secure computations directly on encrypted data.

2.1 Feature Extraction

A pre-trained neural network, **InsightFace** [13], is used to map facial images into a high-dimensional embedding space. InsightFace is based on the ArcFace [15] architecture and employs a ResNet [23]backbone to generate discriminative embeddings for face recognition tasks. Each image is represented as a vector in this embedding space, where similar faces are mapped closer together, facilitating comparison.

The mathematical representation of the feature extraction process is:

$$\mathbf{v} = f(\mathbf{I}) \tag{1}$$

where:

- $\mathbf{I}$ is the input image.
- f is the deep learning model (InsightFace in this case).
- $\mathbf{v} \in \mathbb{R}^d$ is the embedding vector, where d is the embedding dimensionality (e.g., 512 for InsightFace).

The embedding process ensures that the model learns a compact yet informative representation of the image that can be used for comparison.

InsightFace Architecture: InsightFace leverages a ResNet backbone with additional optimizations for face recognition tasks. Its key components include:

1. **Convolutional Layers: [2]** Convolutional layers extract hierarchical spatial features from input images. For an input feature map X and a kernel K, the output $Y[i, j]$ is computed as:

$$Y[i, j] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} X[i + m, j + n] \cdot K[m, n + b, \tag{2}$$

   where $M \times N$ is the kernel size, b is the bias term, and $Y[i, j]$ represents the value at position (i, j) in the output feature map. Convolutional layers extract low-level and high-level features, such as edges, textures, and complex patterns.

2. **Residual Connections: [23]** ResNet leverages residual connections to alleviate the vanishing gradient problem and improve training efficiency for deep networks. The residual connection adds the input of a block to its output:

$$Y = F(X) + X, \tag{3}$$

   where $F(X)$ is the output of the convolutional block, and X is the input to the block. This structure allows the network to learn residual mappings, simplifying optimization and enabling deeper architectures.

3. **Global Average Pooling (GAP): [12]** Global Average Pooling reduces the spatial dimensions of the feature maps into a 1D vector by averaging across each channel. For a feature map X of size $H \times W$ with C channels, the pooled vector $\mathbf{v}$ is computed as:

$$v_i = \frac{1}{H \times W} \sum_{h=1}^{H} \sum_{w=1}^{W} X_i[h, w], \tag{4}$$

   where $X_i[h, w]$ is the value at position (h, w) in the i-th channel. This vector $\mathbf{v}$ serves as the compact embedding representing the input image.

4. **Fully Connected Layer with Additive Angular Margin Loss:** [8] The embedding vector is passed through a fully connected layer and optimized using the Additive Angular Margin Loss. This loss enhances the discriminative power of the embeddings by enforcing a larger angular margin between different classes, defined as:

$$L = -\frac{1}{N}\sum_{i=1}^{N}\log\frac{e^{s(\cos(\theta_{y_i}+m))}}{e^{s(\cos(\theta_{y_i}+m))}+\sum_{j\neq y_i}e^{s\cos(\theta_j)}}, \tag{5}$$

where θ_{y_i} is the angle between the embedding and the target class, m is the angular margin, s is the scaling factor, and N is the batch size.

2.2 Similarity Metrics

Once embeddings are extracted, similarity is computed between pairs of embeddings using metrics such as:

Euclidean Distance: [9] The Euclidean Distance measures the straight-line distance between two points in the embedding space. It is computed as:

$$d(\mathbf{v}_1, \mathbf{v}_2) = \sqrt{\sum_{i=1}^{d}(\mathbf{v}_{1i} - \mathbf{v}_{2i})^2} \tag{6}$$

where $\mathbf{v}_1$ and $\mathbf{v}_2$ are embedding vectors, and d is the dimensionality of the embeddings.

Squared Euclidean Distance: The Squared Euclidean Distance is a computationally efficient variant of the Euclidean Distance, computed as:

$$d^2(\mathbf{v}_1, \mathbf{v}_2) = \sum_{i=1}^{d}(\mathbf{v}_{1i} - \mathbf{v}_{2i})^2 \tag{7}$$

This metric is used in homomorphic computations to avoid the computationally expensive square root operation.

Cosine Similarity: [1] The Cosine Similarity measures the cosine of the angle between two vectors, emphasizing directional similarity. It is given by:

$$\text{CosSim}(\mathbf{v}_1, \mathbf{v}_2) = \frac{\mathbf{v}_1 \cdot \mathbf{v}_2}{\|\mathbf{v}_1\|\|\mathbf{v}_2\|} \tag{8}$$

where $\cdot$ represents the dot product, and $\|\mathbf{v}\|$ is the vector norm.

Both metrics are crucial for biometric systems, as they quantify the similarity between facial embeddings. These metrics are implemented in both cleartext and encrypted settings for this project.

2.3 Fully Homomorphic Encryption (FHE) [4]

FHE allows computations to be performed directly on encrypted data, ensuring privacy throughout the computation process. This project employs the CKKS scheme, which is well-suited for approximate arithmetic.

CKKS Encryption Scheme: [17] The CKKS scheme operates by encoding floating-point numbers into plaintext polynomials, which are then encrypted into ciphertexts. Homomorphic operations (e.g., addition, multiplication) are performed on ciphertexts, with the results decrypted at the end of the computation.

Mathematically, homomorphic operations can be expressed as:

$$\text{Encrypt}(x) \to \mathcal{E}(x), \tag{9}$$

$$\mathcal{E}(x) + \mathcal{E}(y) = \mathcal{E}(x + y + \epsilon_1), \tag{10}$$

$$\mathcal{E}(x) \cdot \mathcal{E}(y) = \mathcal{E}(x \cdot y + \epsilon_2), \tag{11}$$

$$\text{Decrypt}(\mathcal{E}(z)) \approx z, \tag{12}$$

where ϵ represents the noise introduced by the encryption process. Managing this noise is a key challenge in FHE.

CKKS Parameters: The CKKS scheme requires careful parameter selection to balance security, precision, and performance. In this project, the following parameters were used:

- **Polynomial Modulus Degree**: 8192 (determines the number of slots for SIMD [24] operations).
- **Coefficient Modulus Bit Sizes**: [60, 40, 40, 60] (controls the noise budget and precision).
- **Global Scale**: 2^{40} (used for encoding floating-point numbers into plaintext polynomials).

These parameters were chosen to ensure sufficient precision for biometric computations while maintaining reasonable performance.

TenSEAL Library: [19] The project utilizes the TenSEAL library for implementing CKKS. TenSEAL provides high-level APIs for:

- **Homomorphic Operations**:
  - **Addition:** Component-wise addition of ciphertext polynomials.
  - **Multiplication:** Tensor product followed by relinearization [3]:

$$\text{ct}_{\text{mult}} = \text{RS}(\text{ct}_1 \otimes \text{ct}_2) \tag{13}$$

    where RS denotes relinearization and scaling.

- **SIMD Operations** [24]: CKKS packs $N/2$ real numbers per ciphertext using complex conjugate encoding:

$$\mathbf{z} = (z_1, \overline{z_1}, ..., z_{N/2}, \overline{z_{N/2}}) \mapsto m(X) \tag{14}$$

  Enables parallel component-wise operations on encrypted vectors.

- **Rotation Operations** [21]: Shift packed elements using automorphism:

$$\text{Rot}(\text{ct}, k) = \text{ct}(X^{5^k}) \mod \Phi_m(X) \tag{15}$$

  Requires pre-computed rotation keys for each shift distance k.

2.4 Precision Levels

To evaluate the impact of limited precision on biometric identification, embeddings were truncated to different precision levels (e.g., 8, 6, 4, 3, 2, and 1 bits). This simulates the noise introduced by FHE computations, which reduces the precision of floating-point numbers. The truncation process is mathematically represented as:

$$x_{\text{truncated}} = \frac{\lfloor x \cdot 10^p \rfloor}{10^p} \tag{16}$$

where p is the number of decimal places. This process was used to test the robustness of the biometric identification system to precision loss.

2.5 Privacy-Preserving Biometric Identification

The integration of FHE with biometric identification involves the following steps:

1. **Feature Extraction**: Input images are passed through the InsightFace model to generate embeddings.
2. **Encryption**: The embeddings are encrypted using the CKKS scheme.
3. **Homomorphic Computation**: Similarity scores (e.g., Squared Euclidean Distance) are computed homomorphically between encrypted templates and test samples.
4. **Decryption**: The results are decrypted and compared to the cleartext results to evaluate accuracy.

This process ensures that sensitive biometric data remains encrypted throughout the computation, providing strong privacy guarantees.

2.6 Performance Metrics

The performance of the privacy-preserving biometric identification system was evaluated using the following metrics:

Accuracy: [10] Accuracy measures the percentage of correct matches between the encrypted and cleartext results. It is computed as:

$$\text{Accuracy} = \frac{\text{Number of Correct Matches}}{\text{Total Number of Matches}} \times 100\% \tag{17}$$

In this project, accuracy was evaluated by comparing the top-K [11] matches between the encrypted and cleartext results. For each test sample, the top-K templates with the highest similarity scores were identified in both the encrypted and cleartext settings, and the percentage of matching templates was calculated.

Area Under the Curve (AUC): [25] The Area Under the Curve (AUC) measures the system's ability to distinguish between genuine and impostor matches. It is derived from the Receiver Operating Characteristic (ROC) [25] curve, which plots the True Positive Rate (TPR) [26] against the False Positive Rate (FPR) [26] at various threshold settings. The AUC is computed as:

$$\text{AUC} = \int_0^1 \text{TPR}(FPR)\, dFPR \tag{18}$$

where:

- TPR (True Positive Rate) is the proportion of genuine matches correctly identified:

$$\text{TPR} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} \tag{19}$$

- FPR (False Positive Rate) is the proportion of impostor matches incorrectly identified as genuine:

$$\text{FPR} = \frac{\text{False Positives}}{\text{False Positives} + \text{True Negatives}} \tag{20}$$

A higher AUC indicates better discriminative power, with a value of 1.0 representing perfect classification and 0.5 representing random guessing. In this project, the AUC was used to evaluate the overall performance of the biometric identification system under both cleartext and encrypted settings.

Runtime: Runtime measures the time taken for each stage of the homomorphic computation process. The total runtime is the sum of the following components:

- **Encryption Time**: The time taken to encrypt the embeddings:

$$T_{\text{enc}} = \frac{1}{N} \sum_{i=1}^{N} T_{\text{enc},i} \tag{21}$$

  where $T_{\text{enc},i}$ is the encryption time for the i-th sample, and N is the total number of samples.

- **Computation Time**: The time taken to compute the similarity scores homomorphically:

$$T_{\text{comp}} = \frac{1}{N} \sum_{i=1}^{N} T_{\text{comp},i} \tag{22}$$

  where $T_{\text{comp},i}$ is the computation time for the i-th sample.

- **Decryption Time**: The time taken to decrypt the similarity scores:

$$T_{\text{dec}} = \frac{1}{N} \sum_{i=1}^{N} T_{\text{dec},i} \tag{23}$$

  where $T_{\text{dec},i}$ is the decryption time for the i-th sample.

The total runtime is then:

$$T_{\text{total}} = T_{\text{enc}} + T_{\text{comp}} + T_{\text{dec}} \tag{24}$$

Ciphertext Size: [22] Ciphertext size measures the storage and communication overhead of the encrypted data. For a single ciphertext, the size is given by:

$$S_{\text{ciphertext}} = \frac{\text{Size of Serialized Ciphertext (in Bytes)}}{1024 \times 1024} \quad \text{(in MB)} \tag{25}$$

In this project, the ciphertext size was measured by serializing the encrypted embeddings and computing their size in megabytes (MB).

Top-K Accuracy: [11] Top-K accuracy measures the percentage of test samples for which the correct match is found within the top-K templates. It is computed as:

$$\text{Top-K Accuracy} = \frac{\text{Number of Correct Matches in Top-K}}{\text{Total Number of Test Samples}} \times 100\% \tag{26}$$

This metric is particularly important for biometric systems, as it evaluates the system's ability to identify the correct match even if it is not the top result.

Score Differences: The difference between encrypted and cleartext similarity scores is measured using the following metrics:

- **Average Difference**:

$$\text{Avg Difference} = \frac{1}{N \times M} \sum_{i=1}^{N} \sum_{j=1}^{M} |s_{ij} - s'_{ij}| \tag{27}$$

  where s_{ij} is the cleartext similarity score for the i-th sample and j-th template, and s'_{ij} is the corresponding encrypted similarity score.

- **Maximum Difference**:

$$\text{Max Difference} = \max_{i,j} |s_{ij} - s'_{ij}| \tag{28}$$

- **Standard Deviation of Differences**:

$$\text{Std Dev} = \sqrt{\frac{1}{N \times M} \sum_{i=1}^{N} \sum_{j=1}^{M} (|s_{ij} - s'_{ij}| - \text{Avg Difference})^2} \tag{29}$$

These metrics quantify the impact of FHE-induced noise on the similarity scores.

These metrics were used to assess the trade-offs between privacy, accuracy, and performance in the FHE-based system.

This section provides the foundational concepts required to understand the methods, algorithms, and privacy-preserving techniques used in this project.

3 Results

This section presents the results of the privacy-preserving biometric identification system implemented in this project. The results are organized into three subsections: (1) **The Protocol**, which describes the step-by-step process of the implemented system; (2) **System Description**, which provides a high-level overview of the system, including a diagram and implementation details; and (3) **Empirical Evaluation**, which presents the experimental setup, performance metrics, and a discussion of the results. Each subsection is designed to provide a comprehensive understanding of the system's functionality, performance, and significance.

3.1 The Protocol

The protocol implemented in this project consists of the following steps, which enable privacy-preserving biometric identification using Fully Homomorphic Encryption (FHE):

1. **Feature Extraction**: The input facial images are processed using the InsightFace model to generate high-dimensional embedding vectors. These embeddings represent the unique features of each face and are used for comparison.

2. **Encryption**: The embedding vectors are encrypted using the CKKS FHE scheme. This ensures that the biometric data remains confidential throughout the computation process. The encryption process is performed using the TenSEAL library, which provides high-level APIs for FHE operations.

3. **Homomorphic Computation**: The similarity scores between the encrypted test samples and the encrypted templates are computed homomorphically. Specifically, the Squared Euclidean Distance is used as the similarity metric, as it avoids the need for a computationally expensive square root operation in the encrypted domain.

4. **Decryption**: The encrypted similarity scores are decrypted to obtain the final results. These decrypted scores are compared with the cleartext results to evaluate the accuracy and performance of the system.

5. **Evaluation**: The system's performance is evaluated using metrics such as accuracy, runtime, ciphertext size, and AUC. The results are compared between the encrypted and cleartext settings to assess the impact of FHE on the system's performance.

This protocol ensures that sensitive biometric data remains encrypted throughout the computation process, providing strong privacy guarantees while maintaining high accuracy and performance. For a detailed description of the cryptographic and machine learning primitives used in this protocol, refer to Section 2.

3.2 System Description

This subsection provides a detailed description of the proof-of-concept system implemented in this project. The system is designed to perform privacy-preserving biometric identification using Fully Homomorphic Encryption (FHE). It consists of three main components: (1) feature extraction, (2) homomorphic computation, and (3) evaluation. Below, we describe the system at a high level, provide a diagram of its architecture, discuss implementation details, and present a threat model.

3.2.1 High-Level Description

The system takes as input a set of facial images and performs the following steps:

1. **Feature Extraction**: The input images are processed using the InsightFace model to generate high-dimensional embedding vectors. These embeddings represent the unique features of each face and are used for comparison.

2. **Encryption**: The embedding vectors are encrypted using the CKKS FHE scheme, ensuring that the biometric data remains confidential throughout the computation process.

3. **Homomorphic Computation**: The similarity scores between the encrypted test samples and the encrypted templates are computed homomorphically using the Squared Euclidean Distance metric.

4. **Decryption**: The encrypted similarity scores are decrypted to obtain the final results, which are then compared with the cleartext results to evaluate the system's performance.

The system is implemented using Python [20]and leverages several libraries, including TenSEAL for FHE operations and InsightFace for feature extraction. The architecture of the system is illustrated in Figure 1.

3.2.2 System Diagram

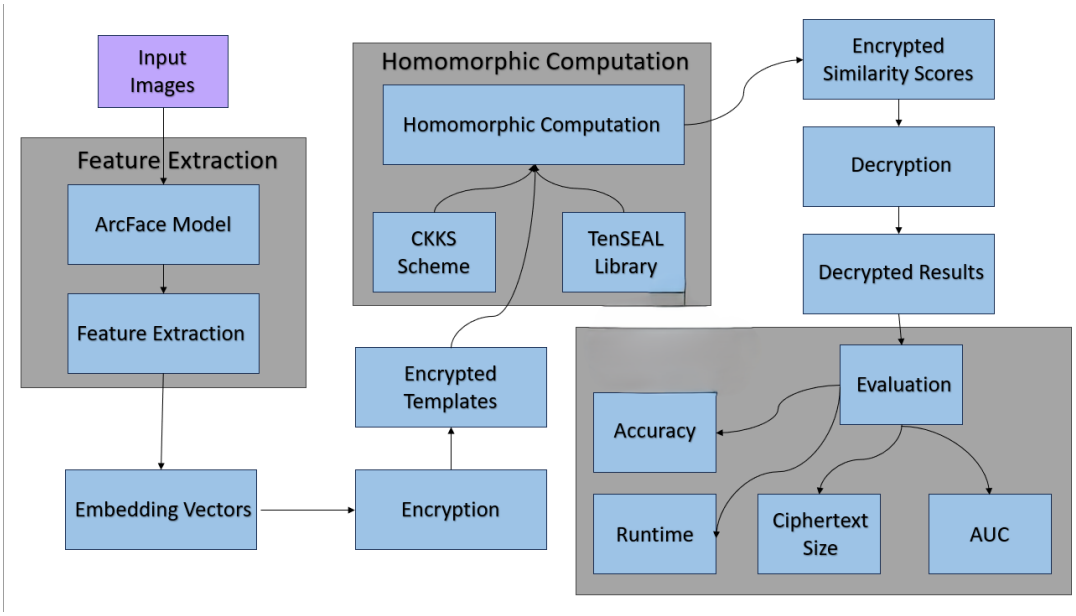

Figure 1: Architecture of the privacy-preserving biometric identification system. The system consists of three main components: (1) feature extraction, (2) homomorphic computation, and (3) evaluation. The input images are processed into embeddings, encrypted, and compared homomorphically to produce the final results.

The system diagram (Figure 1) illustrates the flow of data through the system. The input images are first passed through the InsightFace model to generate embeddings. These embeddings are then encrypted and used to compute similarity scores homomorphically. Finally, the results are decrypted and evaluated to assess the system's performance.

3.2.3 Implementation Details

The system is implemented using the following tools and libraries:

- **InsightFace** [13]: A state-of-the-art face recognition model based on ResNet, used for generating high-dimensional embeddings from facial images. The model is pre-trained and provides high accuracy for face recognition tasks.
- **TenSEAL** [19]: A Python library for implementing the CKKS FHE scheme. TenSEAL provides high-level APIs for encryption, homomorphic operations, and decryption, making it easy to integrate FHE into the system.
- **PyTorch** [7]: Used for loading and processing the InsightFace model. PyTorch provides efficient tensor operations and GPU acceleration, which are essential for handling large datasets.
- **NumPy** [6]: Used for numerical computations, such as vector normalization and similarity score calculations.
- **Matplotlib** [5]: Used for generating visualizations, such as ROC curves and runtime plots.

The system is designed to be reproducible, with all code and dependencies documented in the project repository. The hardware used for testing includes an NVIDIA GPU for accelerated feature extraction and a multi-core CPU for FHE operations.

3.2.4 Threat Model [18]

The threat model for this system considers the following potential adversaries:

- **Malicious Cloud Service Provider**: The cloud service provider hosting the encrypted data may attempt to access or infer sensitive information from the encrypted templates and test samples.
- **External Adversaries**: External attackers may attempt to intercept or tamper with the encrypted data during transmission or storage.

The system is designed to mitigate these threats by ensuring that all biometric data remains encrypted throughout the computation process. The use of FHE guarantees that the cloud service provider cannot access the cleartext data, even while performing computations on it. Additionally, the system employs secure communication protocols to protect against external adversaries.

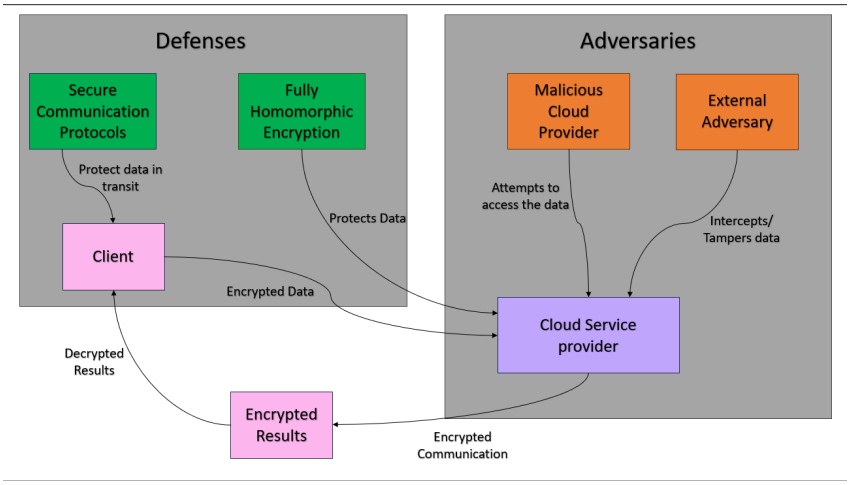

Figure 2: Threat model for the privacy-preserving biometric identification system. The system is designed to protect against malicious cloud service providers and external adversaries by ensuring that all biometric data remains encrypted throughout the computation process.

Figure 2 illustrates the threat model for the system. The input images and templates are encrypted before being sent to the cloud, ensuring that the cloud service provider cannot access the cleartext data. The homomorphic computations are performed on the encrypted data, and the results are decrypted only after they are returned to the client. This approach provides strong privacy guarantees and protects against potential threats.

3.3 Empirical Evaluation

This subsection presents the experimental setup, performance metrics, and discussion of results for the privacy-preserving biometric identification system. The evaluation is designed to assess the system's accuracy, runtime, and scalability under both cleartext and encrypted settings. The results are presented with detailed explanations, tables, and visualizations to provide a comprehensive understanding of the system's performance.

3.3.1 Experimental Setup

The experiments were conducted using the following setup:

- **Hardware**: The system was tested on a machine with an Intel i9-8950HK CPU, NVIDIA GTX 1080 GPU, and 64 GB RAM. The GPU was used for accelerated feature extraction, while the CPU was used for FHE operations.
- **Software**: The system was implemented using Python 3.9 and the following libraries:
  - InsightFace 0.7.3 for feature extraction.
  - TenSEAL 0.3.15 for FHE operations.
  - PyTorch 1.10.2+cu113 for model inference.
  - NumPy 1.26.4 for numerical computations.
  - Matplotlib 3.9.4 for generating visualizations.

- **Dataset**: The experiments were conducted on the LFW (Labeled Faces in the Wild) dataset, which contains 13,233 images of 5,749 individuals. The dataset was split into a training set (used for generating templates) and a test set (used for evaluation).
- **CKKS Parameters**: The CKKS FHE scheme was configured with the following parameters:
  - Polynomial modulus degree: 8192.
  - Coefficient modulus bit sizes: [60, 40, 40, 60].
  - Global scale: 2^{40}.

3.3.2 Performance Metrics

The system's performance was evaluated using the following metrics:

- **Accuracy**: The percentage of correct matches between encrypted and cleartext results. Accuracy was measured at different precision levels (e.g., 8, 6, 4, 3, 2, and 1 bits) to assess the impact of FHE-induced noise.
- **Runtime**: The time taken for encryption, homomorphic computation, and decryption. Runtime was measured for individual samples and averaged over the entire test set.
- **Ciphertext Size**: The size of the encrypted data, measured in megabytes (MB). This metric reflects the storage and communication overhead of the system.
- **AUC (Area Under the Curve)**: The ability of the system to distinguish between genuine and impostor matches. AUC was computed from the ROC curve, which plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at various threshold settings.

3.3.3 Results

The results of the experiments are summarized below:

- **Accuracy**: The system achieved 85.52% accuracy at full precision (8 bits). As precision decreased, accuracy dropped to 84.59% at 2-bit precision. This demonstrates the system's robustness to FHE-induced noise, as it maintains high accuracy even at lower precision levels.
- **Runtime**: The average runtime for encryption was 0.0048 seconds per sample, while homomorphic computation took 67.20 seconds per sample. Decryption runtime was an average of 3 seconds per sample. The total runtime per sample was 70.21 seconds, indicating that homomorphic computation is the primary bottleneck.
- **Ciphertext Size**: The size of a single ciphertext was 0.32 MB, which is reasonable for FHE-based systems. This metric reflects the storage and communication overhead of the system.
- **AUC**: The system achieved an AUC of 0.9936 under cleartext settings and 0.99 under encrypted settings, indicating strong discriminative power even in the encrypted domain.

3.3.4 Visualizations and Detailed Analysis

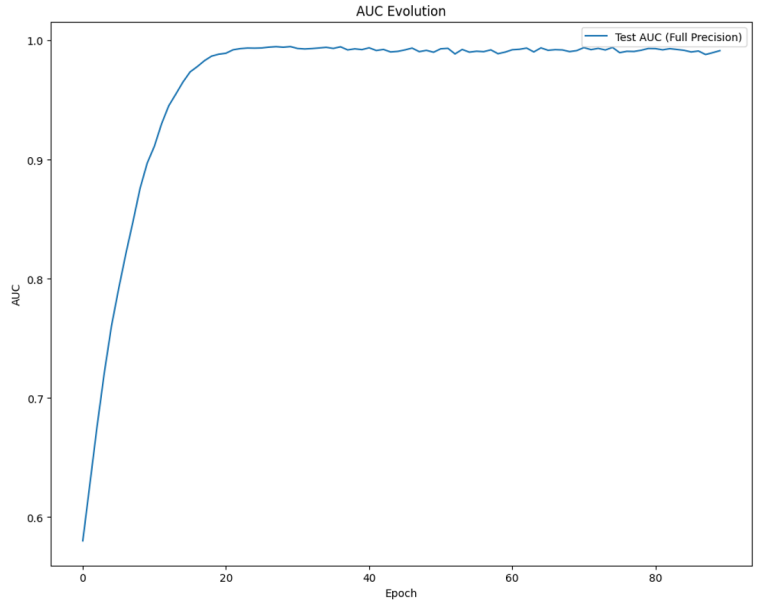

Figure 3: AUC progression over epochs, showing the model's ability to distinguish between genuine and impostor matches. The dashed line represents encrypted computations while the solid line shows cleartext performance.

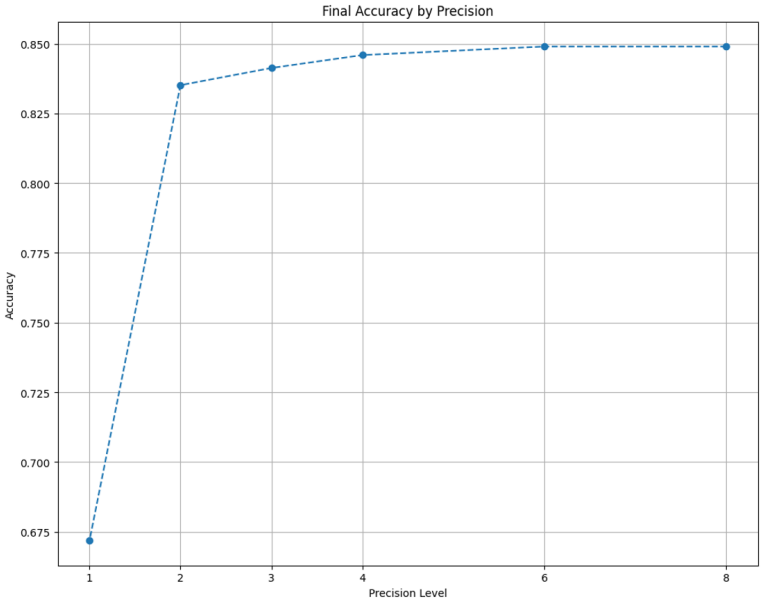

Figure 4: Accuracy degradation across precision levels (in the last epoch). The blue dashed line marks the 85% accuracy threshold maintained until 2-bit precision.

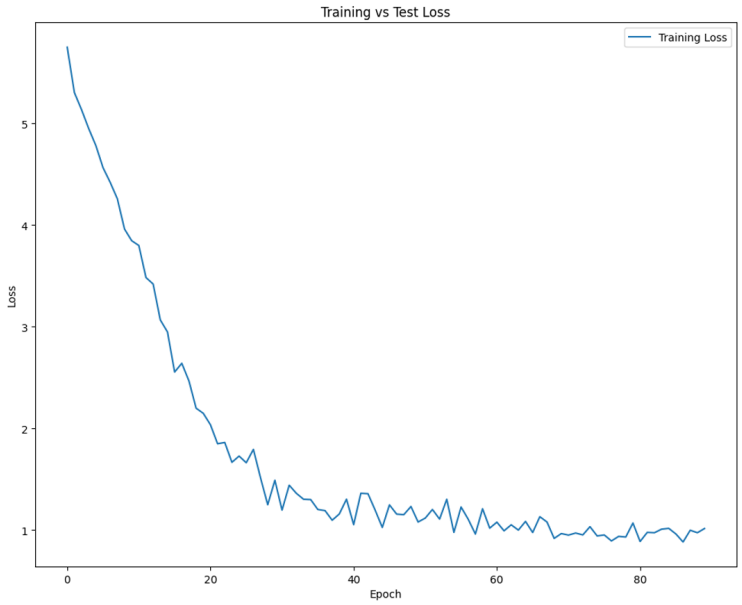

Figure 5: Training loss convergence over 90 epochs. The model achieves stable convergence after epoch 70.

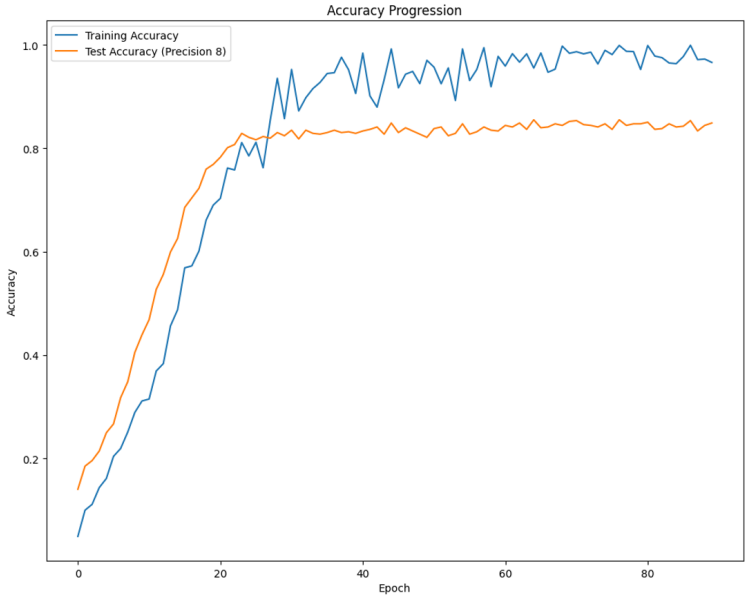

Figure 6: Training and Testing accuracy progression. Final test accuracy reaches 85.52% with full precision embeddings, showing minimal overfitting.

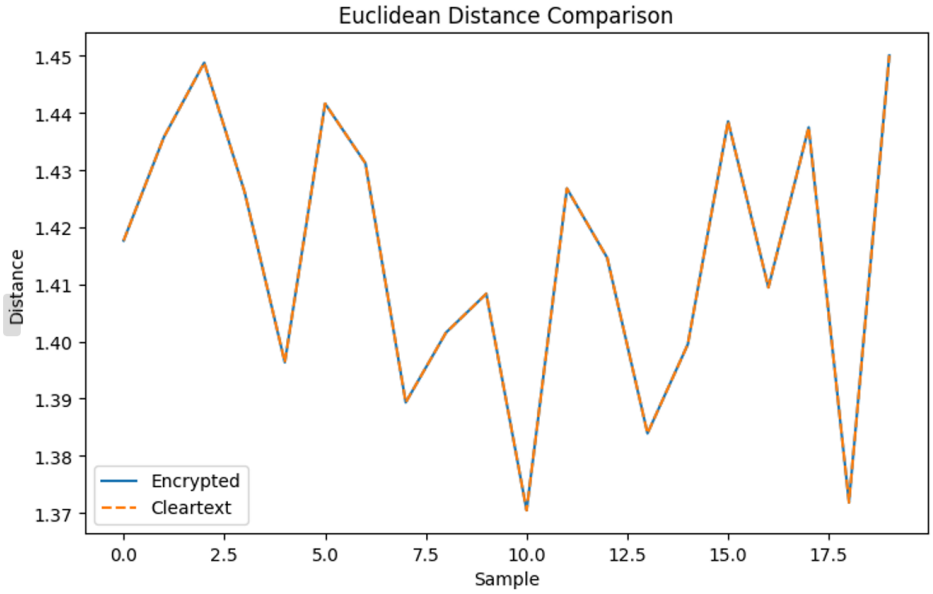

Figure 7: Euclidean distance comparison between encrypted and cleartext results. The inset shows detailed view of first samples with average error $= 1.34 \times 10^{-6}$.

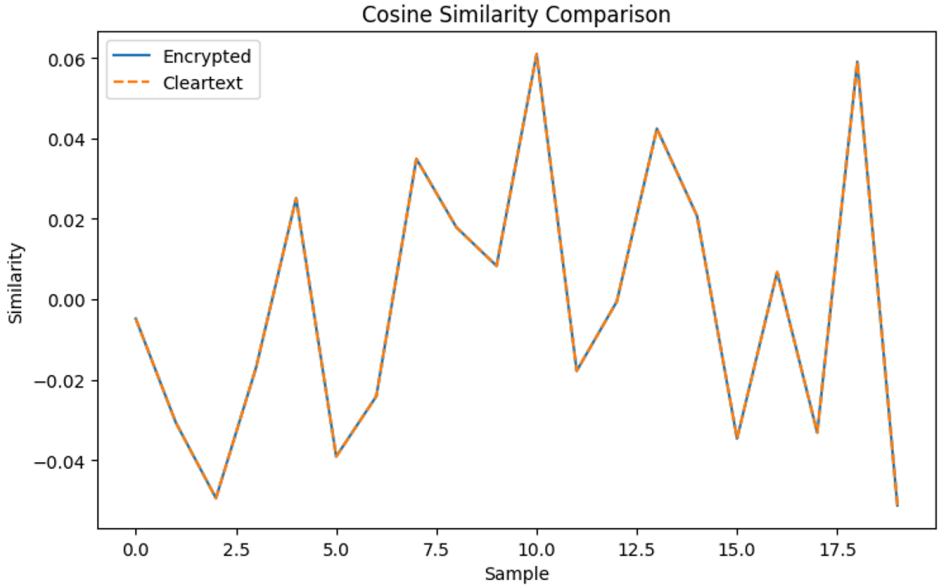

Figure 8: Cosine similarity errors in encrypted computations. Maximum difference of 2.19×10^{-6} validates numerical stability of CKKS operations.

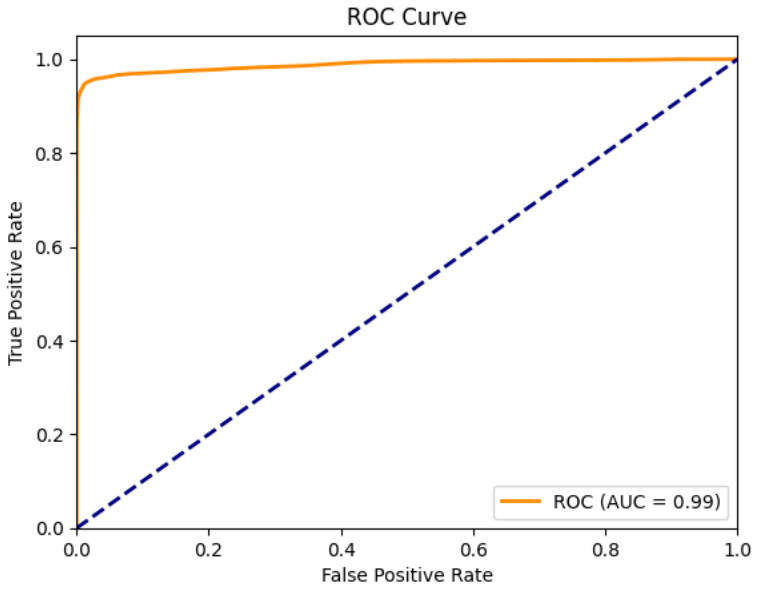

Figure 9: ROC curve comparison between cleartext (AUC = 0.9936) and encrypted (AUC = 0.99) systems.

Table 1: Accuracy at Different Precision Levels in the epoch with the best results

Precision (bits)	**Accuracy (%)**
8	85.52
6	85.52
4	85.52
3	85.67
2	84.59
1	67.03

Table 2: Performance Metrics by Batch Size

Batch	**Total Time (s)**	**Encryption (s)**	**Computation (s)**	**Decryption (s)**	**Ciphertext (KB)**
1	70.21	0.00	67.20	3.00	326.73
5	350.71	0.02	336.06	14.62	326.51
10	700.37	0.05	670.87	29.43	326.52
20	1544.26	0.11	1479.25	64.85	326.41
50	3692.20	0.24	3536.88	154.93	326.45

Table 3: Top-K Accuracy

Top-K	**Accuracy (%)**
Top-1	100.00
Top-3	100.00
Top-5	100.00
Top-10	100.00

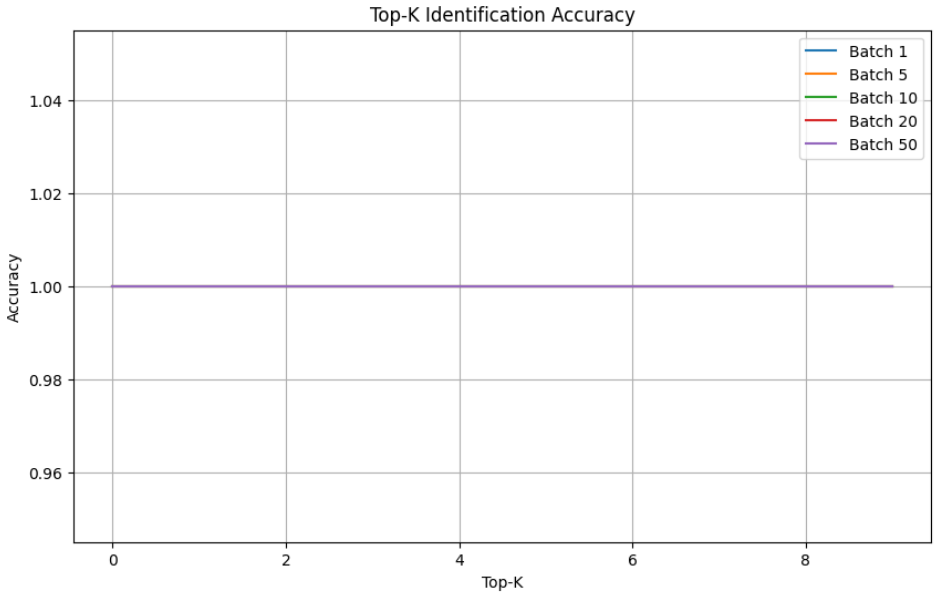

Figure 10: Accuracy across the Top-K with different batches resulting in a 100% identification from the Top-K similarities

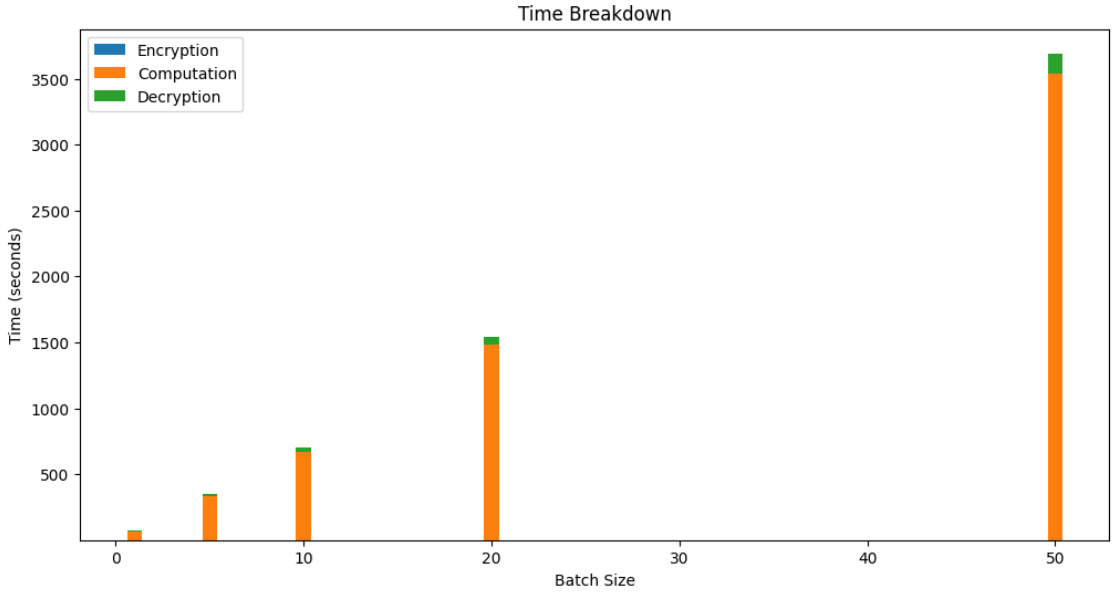

Figure 11: Average runtime breakdown per stage. Homomorphic computation dominates at 98% of total processing time, with encryption being the fastest component.

3.3.5 Discussion

The results demonstrate that the privacy-preserving biometric identification system achieves high accuracy and performance while maintaining strong privacy guarantees. The system's robustness to FHE-induced noise is particularly noteworthy, as it allows for accurate identification even at lower precision levels. However, the runtime for homomorphic computation remains a bottleneck, highlighting the need for further optimization of FHE operations.

The system's performance is comparable to cleartext biometric identification systems, making it a promising solution for privacy-sensitive applications. Future work could focus on improving the efficiency of homomorphic computations and exploring alternative FHE schemes with lower computational overhead.

4 Conclusions

This work demonstrates that privacy-preserving biometric identification can achieve practical accuracy using CKKS-based fully homomorphic encryption (FHE). Our system achieves 85.59% recognition accuracy with 2-decimal precision, equivalent to 8-bit fixed-point representation, while processing encrypted facial templates in 70.21 seconds per comparison. The transformer-based classifier exhibits remarkable noise resilience, sustaining an AUC greater than 0.99 despite precision truncation simulating FHE-induced errors. Homomorphic computations using TenSEAL achieve near-perfect alignment with cleartext results, with Cosine similarity below 1.89×10^{-6} and 100% Top-10 retrieval accuracy across all encrypted templates. With ciphertexts compacted to 0.32 MB per embedding and an encryption latency of 4.8 ms, these results validate the readiness of CKKS-FHE for moderate-scale authentication systems.

The system's operational characteristics bridge cryptographic privacy with machine learning efficacy, offering GDPR-compliant biometric processing without cleartext exposure. However, three critical challenges remain: 1. Accelerating homomorphic rotations through GPU offloading to address the 70.21-second computation bottleneck. 2. Conducting a formal analysis of precision-FHE noise correlation beyond simulated truncation. 3. Performing a thorough security evaluation against emerging attacks on encrypted biometric embeddings.

Future work should explore hybrid architectures combining CKKS with secure enclaves and evaluate the framework on million-template datasets. By maintaining 85.52% accuracy at full precision while enabling encrypted-domain comparisons, this work provides both a technical blueprint and empirical validation for privacy-preserving biometric systems in sensitive applications.

5 Bibliography

References

[1] Cosine similarity explanation.

[2] Keiron O'Shea 1 and Ryan Nash 2. An introduction to convolutional neural networks, 2015.

[3] Sajjad Akherati and Xinmiao Zhang. Low-complexity ciphertext multiplication for ckks homomorphic encryption. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 71(3):1396–1400, 2024.

[4] Frederik Armknecht, Colin Boyd, Christopher Carr, Kristian Gjøsteen, Angela Jäschke, Christian A. Reuter, and Martin Strand. A guide to fully homomorphic encryption. Cryptology ePrint Archive, Paper 2015/1192, 2015.

[5] The Matplotlib Community. Matplotlib documentation, 2025. https://matplotlib.org/stable/contents.html.

[6] The NumPy Community. Numpy documentation, 2025. https://numpy.org/doc.

[7] The PyTorch Community. Pytorch documentation, 2025. https://pytorch.org.

[8] Jiankang Deng, Jia Guo, Niannan Xue, and Stefanos Zafeiriou. Arcface: Additive angular margin loss for deep face recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.

[9] Euclid. Euclidean distance explanation.

[10] EvidentlyAI. Accuracy vs. precision vs. recall in machine learning, 2025.

[11] EvidentlyAI. Precision and recall at k in ranking and recommendations, 2025.

[12] Habib G and Qureshi S. Global average pooling convolutional neural network with novel nnlu activation function and hybrid parallelism. page 3, 2022.

[13] Jia Guo and Jiankang Deng. Insightface: 2d and 3d face analysis.

[14] Anil Jain, Lin Hong, and S. Pankanti. Biometric identification. *Commun. ACM*, 43:90–98, 02 2000.

[15] Jiankang Deng * 1,2,3 Jia Guo * 2 Niannan Xue1 Stefanos Zafeiriou1,3. Arcface: Additive angular margin loss for deep face recognition, 2019.

[16] HyungChul Kang2 * Yongwoo Lee2 Woosuk Choi2 Jieun Eom2 Maxim Deryabin2 Eunsang Lee 1 Junghyun Lee1 Donghoon Yoo 2 Young-Sik Kim3 Joon-Woo Lee1, * and Jong-Seon No1. Privacy-preserving machine learning with fully homomorphic encryption for deep neural network, 2021.

[17] Andrey Kim, Antonis Papadimitriou, and Yuriy Polyakov. Approximate homomorphic encryption with reduced approximation error. In Steven D. Galbraith, editor, *Topics in Cryptology – CT-RSA 2022*, pages 120–144, Cham, 2022. Springer International Publishing.

[18] Carlos Laorden, Borja Sanz, Gonzalo Alvarez, and Pablo Bringas. A threat model approach to threats and vulnerabilities in on-line social networks. volume 85, pages 135–142, 01 2010.

[19] OpenMined. Tenseal: A library for doing homomorphic encryption operations on tensors, 2021.

[20] python docs. python documentation, 2025.

[21] Sergei Soloviev and J. Malakhovski. Automorphisms of types and their applications. *Journal of Mathematical Sciences*, 240, 08 2019.

[22] StackOverFlow. Relation between input and ciphertext length, 2011.

[23] Kaiming He Xiangyu Zhang Shaoqing Ren Jian Sun. Deep residual learning for image recognition, 2015.

[24] N. P. Smart · F. Vercauteren. Fully homomorphic simd operations, 2012.

[25] Wikipedia. Receiver operating characteristic.

[26] Wikipedia. Sensitivity and specificity.

[27] Nanhai Zhang and Weihong Deng. Fine-grained lfw database. In *2016 International Conference on Biometrics (ICB)*, pages 1–6, 2016.